

Companion demo rehearsal

Everything I tried in the app on 17 September 2026 with the 4B model, written so you can repeat it step by step before the showcase: what to type, what should happen, and what I saw.

Model	Gemma 4 E4B, 4-bit (listed in the app as <code>Gemma 4 e4b IT qat ud (Q4_K_XL)</code>)
Where it runs	Entirely on the GPU, 32K context, about 4.6 GB of video memory
Model files	Split into four parts so GitHub can store them; the app reads them as one model. Checked: loads in 5.7 s, chat at 111 tokens per second, a code question answered correctly.
App build	The build of 17 September prepared for the showcase (night decisions 48–54)
Demo project	<code>docs/showcase/demo-project</code> in the repository (a small Python order-totals tool with tests)

1. Before the demo: checklist

Do these on the demo machine ahead of time, not in front of the audience. Every step here is one-off or quick.

Step	Why
☐ Get the code and the model. Install Git LFS once (<code>git lfs install</code>). The repository also holds the Gemma 4 E2B, so fetch only the demo model: clone with <code>git clone -c "lfs.fetchinclude=models/gemma-4-e4b-it-qat/*" <repository URL></code> , or in an existing clone run <code>git config lfs.fetchinclude "models/gemma-4-e4b-it-qat/*"</code> before <code>git pull</code> . Check that <code>models/gemma-4-e4b-it-qat/</code> holds four files ending <code>-00001-of-00004.gguf ... -00004-of-00004.gguf</code> , about 4 GB together (30 MB, 1.5 GB, 1.4 GB, 1.0 GB), not small text placeholders. The E2B folder then holds placeholders, which the app does not list.	The models are stored with Git LFS. Without it, Git fetches placeholder files instead of the model. Every download counts against GitHub's 10 GiB of LFS bandwidth a month: 3.9 GiB for the demo model, 6.8 GiB for both.
☐ Build and start once, well before. Run <code>.\run.ps1</code> in PowerShell or <code>run.bat</code> in Command Prompt (Windows), or <code>./run.sh</code> (macOS, Linux), from the project folder and wait for it to finish starting. Leave that window open during the demo.	The first run builds the inference runtime (about 10 minutes with CUDA) and the UI. Later starts take seconds.
☐ Plug in the power and close games, video editors and other GPU-heavy apps.	On battery the laptop slows down. The model needs about 4.6 GB of video memory, and other apps can hold 1–2 GB.
☐ Open the app at <code>http://localhost:5173</code> in a wide browser window, so the sidebar stays visible.	A narrow window folds the sidebar behind a menu button.
☐ Load the model from the card at the bottom of the sidebar: choose Gemma 4 E4B and press Load . Wait for READY and <code>32K context</code> .	The first load reads 4 GB from disk. After that, a load took 5.6 s.
☐ Warm up: send one short chat message and ask one question in a code task (flows 2 and 3 below). Then delete those sessions from the sidebar.	You see it working end to end, and the demo starts without first-use delays.
☐ Prepare a fresh demo project: copy <code>docs/showcase/demo-project</code> into a new folder of its own (for example <code>demo-run\project</code>). In the Code tab, open the project picker at the top of the sidebar, choose Open or create a project... and select <code>demo-run</code> .	Demo tasks write files and new folders there. A fresh copy means nothing is left over from rehearsal.
☐ Tidy the sidebar: delete old sessions you don't want on screen.	Old chats from other models show a "Last used with another model" banner.
☐ Set the starting mode under the message box for your first code demo (Ask for questions, Plan for a planned change, Auto for building an app), and start each demo with New task .	A fresh task keeps earlier requests out of the model's history.
☐ Keep the prompts below at hand (copy and paste works). Leave Web search (the globe icon) off.	Everything runs locally, and the prompts below are the ones rehearsed.

READY WHEN

The sidebar card shows **READY** with `32K context`, the Code tab shows your demo project, and a warm-up chat answered at 100+ tokens per second.

I added the demo project through the app's API during the rehearsal, so "Open or create a project..." is the one screen I did not click through myself. After it, the new-task screen should show the project name, its path, and three quick actions (Understand this project, Plan a change, Find a bug).

2. A plain chat

In the **Chat** tab, start a new chat and send:

TYPE EXACTLY

Give me three short tips for writing clear commit messages.

WHAT SHOULD HAPPEN

The answer streams in straight away as a numbered list. Under it: the model name, *First text*, *Output speed* and *Total*.

I saw: first text 0.5 s, 100 tokens per second, done in 1.6 s. In the test suite, nine different chat questions took 0.3–2.3 s each at 113–129 tokens per second (higher is faster).

3. Ask about the code (Ask mode)

In the **Code** tab, press **New task**, pick **Ask** below the message box, and send:

TYPE EXACTLY

What does this project do, and how is the tax applied to an order?

WHAT SHOULD HAPPEN

The agent lists the folders, reads `project/README.md` and `project/orders.py`, then answers. Nothing is changed and nothing asks for approval (Ask mode reads freely).

I saw: five steps, no retries, correct answer: a command-line tool that reads an order from JSON and prints the total; `TAX_RATE = 0.08`; `order_total` multiplies the subtotal by `1 + tax_rate` and rounds to two decimals.

4. Plan a change, keep planning, then approve

Start a new task, pick **Plan**, and send:

TYPE EXACTLY

Add a function `total_quantity(items)` to `project/orders.py` that returns the sum of all item quantities, with a unit test in `project/test_orders.py`.

WHAT SHOULD HAPPEN

The agent reads both files and presents a plan. A card appears: **The plan is ready** — Yes, accept edits · Yes, ask before edits · No, keep planning. Nothing has been changed yet.

1. Press **No, keep planning**. The card closes, the mode stays on **Plan**, and the cursor is back in the message box.
2. Send the refinement:

TYPE EXACTLY

Also run the tests with `python -m unittest` inside the project folder after the change.

A new plan arrives that includes running the tests, with the card again.

3. Press **Yes, accept edits**. The mode switches to **Accept edits** and the agent starts working without asking for file edits.
4. When it wants to run the tests it stops with **Waiting for approval**. Press **Review the request**: the card names the command, `python -m unittest` (in project). Press **Allow once**.

WHAT SHOULD HAPPEN

`orders.py` gains `total_quantity`, `test_orders.py` gains a test, and the tests pass. Each file change has **View changes**, showing only the changed lines.

I saw: the model first forgot to import the new function, so the test run failed with a `NameError`. It read the error, fixed the import and asked to run the tests again (a second **Allow once**). All three tests then passed.

Commands ask every time in Ask and Accept edits, so there is no "Allow for session" for them. To let a whole task run without prompts, use **Auto** (next flow).

5. Build an app from scratch (Auto mode)

Start a new task, pick **Auto**, and send:

TYPE EXACTLY

Create a to-do list web app in a new folder named `todo`: a single `index.html` with its CSS and JavaScript inside. I can add a task, tick it as done, and delete it.

WHAT SHOULD HAPPEN

No approval prompts. The agent creates `todo/index.html`, checks it, and reports. Open that file in your browser: type a task and press **Add Task**; click a task to mark it done; **Delete** removes it; the list survives a reload.

I saw: about 50 s. Everything worked; empty tasks are ignored. On an earlier try, before a fix made that morning, the model wrote the page into the wrong folder. The app now catches that and sends it back.

6. The two test-suite apps

The same way (new task, **Auto**), these are the exact prompts the automated test runs used:

TYPE EXACTLY

Create a basic calculator web app in a new folder named `calculator`: a single `index.html` with its CSS and JavaScript inside. It needs buttons for the digits 0-9, +, -, ×, ÷, a decimal point, clear and equals, and a display that shows what is typed and the result.

TYPE EXACTLY

Create a web app in a new folder named `clock`: a single `index.html` with its CSS and JavaScript inside, with three tabs: a live digital clock, a stopwatch with start, stop and reset, and a countdown timer where the user enters minutes and seconds and presses start.

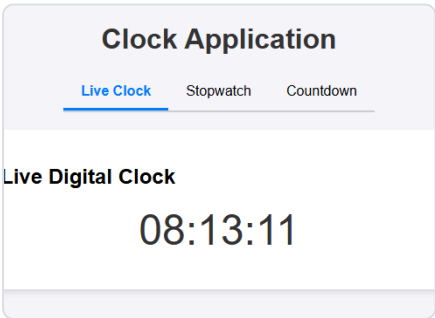
WHAT I SAW

Calculator in 47.8 s: I clicked $7+5=12$, $6\times7=42$, $9\div4=2.25$, $3-8=-5$, $1.5+2.25=3.75$ and $2+3\times4=14$ (correct order of operations); C resets. The model's own slips: a second "=" button, and "/" on the divide key.

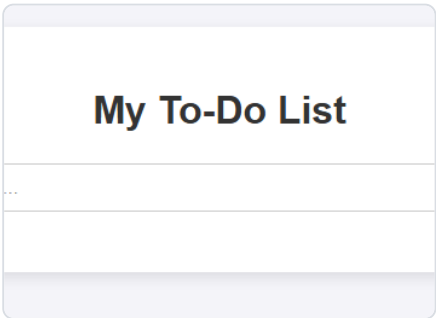
Clock in 54.3 s: styled page, ticking clock, Stopwatch and Countdown tabs.



Calculator (test run)



Clock (test run)



To-do app (rehearsal)

7. Switching modes

- Click **Ask**, **Accept edits**, **Plan** or **Auto** under the message box, or press **Shift** + **Tab** in the message box to cycle Ask → Accept edits → Plan. Auto is left out of the cycle on purpose; pick it with a click.
- Ask: reads freely; edits, commands and deletions ask. Accept edits: reads and file edits run; commands, deletions, Git and web search ask. Plan: read-only, ends with a plan to approve. Auto: carries out the task without prompts.

8. Timings at a glance

What	Time I measured
Load the model	5.6 s
Chat reply	first text 0.1–0.5 s; 100–129 tokens/s; 0.3–2.3 s in total
Question about the project (Ask)	a few seconds of model time, 5 steps
Plan (Plan mode)	about 20 s
Change with tests (test suite, Auto)	16.3 s, 5 tests passing
Calculator / clock / to-do app (Auto)	47.8 s / 54.3 s / about 50 s

9. Rough edges to know about

- **A 4B model still makes small mistakes.** Examples from today: "/" instead of "÷", a forgotten import (it fixed that itself after the test failed), and a docstring edit that removed the `def` line it was documenting. Asking it to run the tests after a change, and looking at **View changes**, shows these quickly.
- **Technical wording in the activity list.** When a reply cannot be read, progress rows mention things like "the constrained JSON format". The run carries on; the wording is not yet plain.
- **New code sessions stay titled "New task"** in the sidebar.
- **The "Last used with another model" banner** in an old chat stays after the next reply.
- **Start each demo in a fresh task.** A long session carries earlier tasks in its history, which a small model can confuse with the new request.

10. Afterwards

Delete the `demo-run` folder (or send it to the Recycle Bin). The app keeps the sessions in its history until you delete them from the sidebar.